

# detonate.sg — A Consumer Scam-Checker That Proves the Telegram GST Voucher Worm Is Real

## TL;DR

- Build a consumer “paste-the-message” scam checker that renders a plain-English verdict AND runs a live “detonation engine”: a Daytona sandbox drives a headless Chromium browser through an Oxylabs Singapore residential proxy to defeat the phishing kit’s geo-cloaking, screenshots the fake Telegram login page, and an ai&-hosted LLM (plus a Nosana-hosted open vision model) explains exactly how the page would hijack your Telegram account and spread to your contacts. This is novel because ScamShield and generic link-checkers give a yes/no verdict from *outside* the cloak; detonate.sg gets *inside* it and shows the evidence.
- The threat is real and current: on 1 October 2025 the Singapore Police Force issued a specific advisory on fraudulent Telegram/Instagram accounts promoting government payouts (SG60/CDC/GST Vouchers) that led to victims’ Telegram accounts being taken over — a self-propagating worm that spreads from trusted contacts.
- Keep the name **detonate.sg** as the engineering/brand handle, but pitch it under a consumer tagline (“See the trap before it springs”). Daytona is load-bearing (isolated execution of an untrusted, likely-malicious link is the *only* safe way to do this), and the safest on-stage demo detonates against a **team-built mock phishing page** that mirrors the real kit — never live criminal infrastructure or a real Telegram account.

## Key Findings

### The scam is officially recognized and dangerous

- The Singapore Police Force published “Police Advisory On Fraudulent Telegram Channels And Instagram Accounts On Government Payouts That Led To The Takeover Of Telegram Accounts” on 1 October 2025. It confirms victims are asked to disclose personal details (mobile number, name, NRIC) and Telegram codes, “which in turn would allow scammers to take over their accounts.”
- The GST Voucher (GSTV) is a **real** Singapore government scheme, which is exactly why the “\$850 up to \$1,000 starting July 2026” lure is convincing. Per MOF’s 9 July 2026 release, “In August 2026, about 1.5 million eligible adult Singaporeans will receive up to

\$850 in cash as part of GSTV – Cash”; those who never signed up before had to sign up by 14 July 2026 and link NRIC to PayNow by 28 July 2026 to be paid by 7 August 2026. The scam message’s dates and dollar figures deliberately shadow this real timeline.

- MOF does **not** have a Telegram broadcast channel and never sends clickable links or asks for authentication codes. Genuine gov accounts carry a blue verification tick.
- **Scale:** Per the SPF Mid-Year Scam and Cybercrime Brief 2025 (released 30 August 2025), S\$456.4M was lost across 19,665 scam cases in Jan–Jun 2025 (down from S\$522.4M in H1 2024). Telegram-involved scam cases had earlier spiked 137.5% in H1 2024 versus H1 2023 (of which 38.6% investment, 29.6% e-commerce, 25% job scams). Per the SPF Annual Scam and Cybercrime Brief 2025 (released 25 February 2026), full-year 2025 saw 37,308 scam cases (–27.6%) and S\$913.1M lost (–17.9%); government-official-impersonation scams (GOIS) “reported more than doubled, by 123.6% to 3,363 cases in 2025, from 1,504 cases in 2024,” with losses up 60.5% to S\$242.9M — the second-highest loss category (26.6%), behind investment scams at S\$336.2M (36.8%). Telegram cases fell in 2025 (all platforms except TikTok saw significant decreases, attributed to Online Criminal Harms Act obligations requiring platforms to disrupt malicious activity; TikTok rose 37.8%) — yet the government-payout Telegram-takeover campaign *resurged* in October 2025, which is what the SPF advisory addresses.

## How the attack works technically (the “worm”)

1. **Lure:** A DM arrives from a *real contact* (whose account is already hijacked) with the GST Voucher message and a shortened link.
2. **Cloaking:** The link (via a URL shortener, which hides the destination and evades anti-phishing filters — Kaspersky) leads to a phishing page that uses geo-fencing/anti-bot cloaking. Modern kits (e.g., the “Blacksite”/Cloaked.gg AiTM kit) serve a decoy or “403 Forbidden” to scanners from datacenter ASNs, and the real phishing page only to targets in the right geography/device. A clean automated verdict on a cloaked URL is therefore *inconclusive, not safe*.
3. **Harvest:** The fake Telegram login page collects phone number, the Telegram login OTP, and the 2FA cloud password, and sends them to the attacker “using an AJAX POST request” (iZOOlogic). The kit pulls attacker-controlled `api_id / api_hash` at runtime to render a convincing login and build a real authorized session (CybersecurityNews/Cryptika).
4. **Takeover:** Attacker registers the credentials/session as a new authorized device — with a reused live session token, it may not even trigger a “new login” warning.
5. **Self-propagation:** The compromised account becomes an automated “secondary

delivery mechanism" (CYFIRMA) that blasts the same phishing link to the victim's contacts, closing the worm loop. "The attacker then sends a similar phishing message from the compromised account to the victim's entire contact list. The cycle repeats, leading to a massive chain of compromised accounts across Telegram" (iZOOlogic). MITRE mapping: T1078 (Valid Accounts) + T1090 (Proxy — using compromised accounts to propagate). The trust of a familiar sender name is the payload.

## Existing tools and the white space

- **ScamShield** (GovTech/SPF/NCPC/OGP) is the incumbent: app + 1799 helpline + website, with an AI classifier that checks suspicious numbers, links and messages (including WhatsApp/Telegram screenshots) and lets users report and block. But it returns a verdict from *outside* — it does not detonate the link inside a sandbox from a Singapore vantage point, does not actively defeat geo-cloaking, and does not show the account-takeover mechanism as evidence.
- **urlscan.io, VirusTotal, Bitdefender/Kaspersky link checkers** scan from cloud/datacenter IPs and are exactly what phishing kits cloak against — they frequently see the benign decoy and return a misleadingly clean result.
- **White space:** A *consumer* tool that (a) speaks plain English, (b) is Singapore-geolocated to beat the cloak, and (c) produces *visceral proof* of the Telegram takeover, not just a risk score. No consumer-facing tool does the "watch it try to steal your login code" demonstration.

## What "ai&" is (sponsor identification — confirmed)

"ai&" is "**ai& Inference**" (aiand.com; console.aiand.com), an AI inference platform that serves frontier **open-weight** models (Kimi, DeepSeek, GLM) via **OpenAI- and Anthropic-compatible APIs**  — you switch by changing the `base_url`. It runs on Japan-based infrastructure (low APAC latency, Japanese data residency; "Your data stays in Japan. We never use your inputs or outputs for model training"), offers pay-as-you-go pricing with no contracts, has a browser Playground, and (per a developer writeup on Qiita) a no-credit-card free trial with promo credits (e.g., \$50). It effectively replaced Kimi/Moonshot as the hackathon's inference sponsor but conveniently still serves Kimi among its models. **Role in the build: the plain-English consumer verdict/explanation generator.**

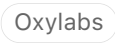
## Details

### Product design (buildable in ~5 hours)

#### User flow (consumer surface):

1. User pastes the suspicious Telegram message (or forwards it to a Telegram bot / web app).
2. The app extracts the URL and instantly returns a provisional verdict from text analysis.
3. In the background, the **detonation engine** runs (~10–40s) and the UI streams: "Visiting the link from a Singapore home connection... following redirects... capturing the page... analyzing..."
4. Final verdict card: **"SCAM — this link will try to hijack your Telegram account and spread to your friends,"** with a screenshot of the fake login page, the harvested-field list (phone / OTP / 2FA password), the redirect chain, and a one-tap "Report to ScamShield (1799)" + "Enable Telegram 2FA" guidance.

#### Detonation engine (the technical core):

- A **Daytona sandbox** is created per check. Untrusted, likely-malicious link execution is the natural, load-bearing reason Daytona is needed: you never run this on your own server or the user's device.
- Inside the sandbox, **headless Chromium via Playwright** launches with the proxy set to an **Oxylabs Singapore residential exit** (`proxy: { server, username, password }`),  so the phishing kit sees a genuine SG home IP and serves the *real* page instead of a decoy.
- Playwright follows the redirect chain, captures a full-page screenshot, and extracts the DOM: form fields, input names/placeholders, any AJAX POST endpoints, and JS that references Telegram `api_id / api_hash` or contact-list propagation.
- **ai& Inference** (OpenAI-compatible call) takes the extracted signals + screenshot description and writes the consumer verdict and explanation ("This page is impersonating Telegram's login. It asks for the code Telegram texts you — if you type it in, criminals log in as you and message everyone in your contacts.").
- **Nosana-hosted open vision model — Qwen2.5-VL-7B-Instruct** (top pick), or **Llama-3.2-11B-Vision-Instruct** as an alternative; both open-weight, on Hugging Face, runnable via vLLM/Ollama in Docker, and fit a single 24GB GPU — performs **brand-impersonation detection from the screenshot**: "Is this a Telegram login page (Telegram logo/branding + phone/OTP form)?" → JSON `{brand, is_login_form, confidence}`. Qwen2.5-VL is preferred for its native dynamic-resolution input (reads a full webpage screenshot without downscaling), strong OCR (reads the URL bar and form labels), and clean structured-JSON output. Research (arXiv:2511.09606) shows VLMs detect impersonated brands best from full screenshots rather than cropped logos, validating this design.

## The safe on-stage demo (critical)

Do **NOT** detonate against live criminal infrastructure on stage (URLs die/change, interacting may be unlawful, and it's unreliable). Do **NOT** complete a real Telegram login (violates ToS, hijacks a real account). **Recommended: detonate against a team-built mock phishing page** that faithfully mirrors the real kit — same Telegram-clone login UI, same phone→OTP→2FA field sequence, same AJAX POST harvest, and a visible client-side "propagate to contacts" code stub. This delivers the visceral "watch it try to steal the login code" moment **safely and reliably**, and you can show the captured POST request payload as evidence without ever completing a login. Optionally also show a *captured-and-replayed* real POST request (recorded earlier, redacted) as corroboration. Frame it explicitly on stage: "This is our own controlled replica of the real kit — we never touch a real victim's account or live criminal servers."

## Sponsor tool mapping (concrete)

- **Daytona (flagship, load-bearing):** Per-check isolated cloud sandbox that runs the untrusted link-visit + Playwright detonation. Sub-90ms creation; [Daytona](#) OCI/Docker images so you can bake in Chromium/Playwright; ephemeral + parallelizable across many checks. Isolation is the *reason* the product is safe — not decorative.
- **Oxylabs:** Residential proxies with **Singapore geo-targeting** (100M+ IP, 195-country network, [ScrapeOps](#) free geo-targeting, [Oxylabs](#) sticky sessions) routed into Playwright's proxy config to defeat the kit's geo-cloaking so the sandbox sees the real phishing page, not a decoy. Documented Playwright integration ( `pr.oxylabs.io:7777` + country entry nodes).
- **ai& Inference:** OpenAI/Anthropic-compatible LLM inference (open-weight Kimi/DeepSeek/GLM) generating the plain-English consumer verdict + explanation. Drop-in via `base_url` change; free trial credits.
- **Nosana:** Decentralized GPU host serving the open vision model (Qwen2.5-VL-7B) for brand-impersonation detection from the screenshot; Docker + vLLM/Ollama + Hugging Face models supported; email-login free credits.

## Known technical constraints — de-risk in first 30 minutes

- **Daytona network egress by tier (top risk):** On **Tier 1 & Tier 2, network access is restricted and cannot be overridden at the sandbox level** — organization-level restrictions take precedence. [Daytona](#) Only **Tier 3/Tier 4** allow full/adjustable outbound internet (and dynamic egress changes on running sandboxes). [Daytona](#) Your detonation needs outbound internet (to reach the link via Oxylabs). **Verify your tier first**; if stuck on Tier 1/2, either upgrade, request a sponsor tier bump/credits, or fall

back to running the Playwright detonation on a permitted host while still using Daytona for the untrusted DOM/JS parsing and any code-analysis step (keeping Daytona load-bearing for isolated execution of untrusted artifacts). Network allow-list is IPv4 CIDR only, max 10 entries `Daytona` — you'd need to allow the Oxylabs gateway IP range.

- **Headless Chromium in a sandbox:** bake Playwright + Chromium into the Daytona snapshot/Docker image to avoid slow cold installs; run `headless` but set a realistic user-agent (some proxies/sites treat headless differently).
- **Oxylabs auth:** use username/password in Playwright `proxy` (HTTP Basic); test with a quick `ip.oxylabs.io` call to confirm the SG exit before wiring the full flow.
- **ai& / Nosana:** confirm API keys + one working chat completion and one working vision inference call in the first 30 minutes; keep a hardcoded fallback verdict template in case either is flaky under venue Wi-Fi.
- **First-30-min checklist:** (1) confirm Daytona tier allows egress; (2) Oxylabs SG proxy returns an SG IP; (3) ai& chat call works via `base_url` swap; (4) Nosana vision endpoint returns JSON; (5) mock phishing page deployed and reachable from the sandbox.

## 2-minute demo script

- **0:00–0:20 (gut punch):** "Last October, Singaporeans started getting Telegram messages from *friends* — real friends — saying 'Claim your \$850 GST Voucher.' The friend was real. The message was a trap. One tap and the scammer becomes you, and messages everyone you know. The police issued a warning. There's still no tool that shows people *why* it's dangerous."
- **0:20–0:40 (framing):** "GST Vouchers are a real government payout — 1.5 million Singaporeans get up to \$850 this August — that's why it works. ScamShield can tell you a link looks risky, but scam pages hide from scanners. So we built `detonate.sg` — it doesn't guess, it *detonates*."
- **0:40–1:30 (live demo):** Paste the message → provisional flag → "Detonating from a Singapore home connection to beat the cloak..." → Daytona sandbox + Oxylabs SG proxy load the (mock) page → screenshot of the fake Telegram login appears → vision model tags "Telegram login impersonation, 98%" → the app highlights the phone/OTP/2FA fields and the contact-propagation code → verdict card: **"SCAM — this steals your Telegram login code and spreads to your contacts."**
- **1:30–2:00 (close):** "Four sponsors, each load-bearing: Daytona safely runs the untrusted link, Oxylabs' Singapore proxy beats the geo-cloak, ai& writes the human explanation, Nosana's GPU vision model spots the fake Telegram brand. It's complete, it solves a real, currently-active Singapore problem, and it's the first consumer tool that

shows you the trap instead of just scoring it. One tap to report to ScamShield 1799. That's detonate.sg."

## Legal/ethical framing (have ready for judges)

- **Scraping/proxies:** We only visit a URL a user *received and voluntarily submitted*, once, to protect that user — analogous to ScamShield's own link-checking and to commercial anti-phishing crawlers (e.g., ZeroFox, CloudSEK) that use residential proxies to defeat cloaking. Oxylabs residential IPs are ethically sourced; we do not scrape at scale or harvest personal data.
- **Interacting with phishing infrastructure:** For the demo we detonate against our **own** replica, not live criminal servers or real Telegram accounts; production would visit only user-submitted links, read-only, in an isolated sandbox, and never submit credentials.
- **Data privacy:** The link and screenshot are processed transiently in an ephemeral sandbox; no victim credentials are ever entered; verdicts can feed the existing ScamShield pipeline rather than a parallel data store.
- **Responsible disclosure:** Detected phishing URLs/screenshots can be forwarded to ScamShield and SingCERT (Telegram takeover incident reporting form) rather than published.

## Recommendations

1. **Lock the architecture in the first 30 minutes** against the checklist above — the single biggest risk is Daytona egress tier. If egress is blocked and can't be upgraded, pivot Daytona to the "isolated untrusted-DOM/JS analysis + code-execution" role and run Playwright egress from a permitted host; keep Daytona load-bearing in the narrative (isolated execution of untrusted artifacts).
2. **Build the mock phishing kit early** — it's both your safe demo target and your reliability insurance. Make it visually convincing (Telegram logo, phone→OTP→2FA flow) and include a visible contact-propagation code stub for the "worm" reveal.
3. **Prioritize the demo moment over feature breadth:** judges score completeness, innovation, real-problem-solving, and sponsor usage. The screenshot-appears + "Telegram impersonation 98%" + field-highlight sequence is the winning 20 seconds — make it smooth and fast, and pre-warm/cache calls for the demo URL.
4. **Wire all four sponsors visibly** and narrate each one's load-bearing role in the pitch (there is an explicit sponsor-usage bonus in the Daytona HackSprint rubric).
5. **Name:** keep **detonate.sg** — memorable, technical, and .sg signals the Singapore focus.

Add the consumer tagline "See the trap before it springs."

6. **Benchmarks that change the plan:** if the SG proxy can't reliably reach real cloaked pages in testing, lean fully on the mock-kit demo and frame live detonation as roadmap. If ai&/Nosana latency exceeds ~15s under venue Wi-Fi, pre-warm calls and cache the vision result for the demo URL. If Daytona egress is unavailable and unfixable, execute the fallback in Rec 1 rather than dropping Daytona.

## Caveats

- The specific shortener "sw.run" in the user's example could not be corroborated in named security-research sources; general URL-shortener abuse in Telegram takeover campaigns is well-documented (Kaspersky, Palo Alto Unit 42). Treat the exact shortener as illustrative.
- Full-year 2025 SPF data shows Telegram scam cases fell in 2025 (after Online Criminal Harms Act platform obligations) even though the government-payout Telegram-takeover campaign resurged in October 2025 — frame the problem as "active and officially warned-about," not "rising on every metric." Government-official-impersonation scams as a category, by contrast, more than doubled (+123.6% cases) in 2025.
- Some propagation-automation details come from vendor/secondary write-ups; the strongest primary/named sources are iZOOlogic, CYFIRMA, Kaspersky, Palo Alto Unit 42, and SPF. The mechanism (AJAX-POST harvest → authorized session via attacker `api_id / api_hash` → automated spread to contacts) is well-supported; the exact automation library varies by campaign.
- Daytona free/low tiers may block the outbound internet the detonation needs — this is the top build risk and must be verified before committing.
- "ai&"/"ai& Inference" is a relatively new, lightly-documented platform; confirm exact model names, endpoints, and free-credit terms in the console at the start of the build, and keep an OpenAI-compatible fallback provider configured (base\_url swap) in case of downtime.